Guion de los expositores

A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting (Euro-Par 2024)

Reparto y tiempos

Parte	Expositor	Diapositivas	Tiempo
Apertura y plan	David	1–3	2 min
1. El problema	**David**	4–14	≈ 8 min
2. La solución	**Hugo**	15–26	≈ 10 min
3. Corrección, evaluación y conclusiones	**Diego**	27–41	≈ 10 min
Preguntas	los tres	41 (y 42–43 si hacen falta)	resto

Total ≈ 30 min. Las diapositivas 42 (referencias) y 43 (anexo) son de reserva.

Antes de empezar

- Abrir `exposicion.pdf` a pantalla completa (diapositiva 1) y comprobar que el pie muestra «Expone: Nombre».
- Quien cambie de parte lo hace en la portadilla (4, 15, 27): es el momento de pasarse el mando.
- Opcional, para repetir las demos en vivo: una terminal en WSL con `bash codigo/ejecutar_pruebas.sh` (desde `codigo/`) y otra con `node simulador/buscar.js 20000 ingenua,sinReserva` (desde `simulador/`). Las capturas de las diapositivas son salidas reales; si la demo falla, se enseña la captura.
- Tener a mano la analogía: **libro** = objeto, **hoja de usuarios** = contador, **pase** = referencia (peso 1), **cartel** = *atomic shared pointer*, **talonario** = peso, **tirar el libro** = liberar.
- Datos clave que conviene saberse de memoria: ASan detecta `heap-use-after-free` en la ingenua y 0 errores en el código de los autores; en el simulador el paper tiene 0 fallos; con 12 hilos y 1 pila, 34,3 Mop/s (autores) frente a 2,2 (GCC).

Parte 1 — David: el problema

Diap. 1 (portada), 2 (plan), 3 (analogía)

- Presentarnos: «Hoy exponemos un paper de Euro-Par 2024 sobre conteo de referencias concurrente.»
- Tres partes: yo el problema, Hugo la solución, Diego la corrección, los experimentos y nuestras pruebas.

- Leer la analogía despacio: es la que usaremos toda la exposición. Cartel = variable compartida; talonario = peso.
- Frase clave: «el cartel lleva un talonario de tickets ya apuntados en la hoja; leer es arrancar un ticket».

Diap. 5: El paper en una frase

- La pregunta: leer un puntero compartido con conteo de referencias *mientras otro hilo lo cambia*, sin candados, sin esperas y sin memoria extra.
- Euro-Par 2024, Utrecht y CMU; tiene sello *Artifact Evaluated*: el código es público y lo hemos usado.
- Mejora el *split reference count* (Folly, GCC), que solo es *lock-free*; el paper lo hace *wait-free* y lo explica más simple.

Diap. 6: Conteo de referencias en 30 segundos

- Copiar = rc += 1; destruir = rc −= 1; rc = 0 ⟶ se libera en ese instante.
- Ventaja: con rc = 1 el dueño modifica en el sitio. Usado en `shared_ptr`, Rust `Arc`, Swift, CPython.
- «El problema aparece cuando varios hilos comparten una *variable* que apunta al objeto.»

Diap. 7: Shared pointer y atomic shared pointer

- *Shared pointer*: inmutable; su contador ya es seguro entre hilos. *Atomic shared pointer*: celda mutable con `load` y `store`.
- En C++20 es `std::atomic<std::shared_ptr<T>>`. Señalar la Fig. 1: el contador junto al objeto (a), el caso que se explica.

Diap. 8: La carrera read-reclaim (la diapositiva importante de mi parte)

- Seguir las flechas: Ana lee el cartel (A) ⟶ el sistema la pausa ⟶ Beto pone B, resta uno a A, rc = 0 y **libera A** ⟶ Ana despierta y suma 1 a un libro que ya no existe.
- Dos pasos (leer puntero, apuntarse) que no son atómicos. La ventana dura nanosegundos: pasa las pruebas y falla en producción.

Diap. 9: ¿Por qué no hacerlo en un solo paso?

- Puntero y contador están en dos sitios distintos de la memoria.
- La tabla: existen `fetch_and_add`, `swap`, CAS y CAS de doble anchura (dos palabras contiguas); **no** existen DCAS ni `fetch_and_add` de 128 bits.
- Conclusión: una atómica abarca una palabra; no puede leer el cartel y tocar la hoja a la vez.

Diap. 10: Garantías de progreso

- Bloqueante: alguien puede esperar siempre. *Lock-free*: siempre *algún* hilo avanza, pero yo podría fallar siempre (bucle CAS).
- *Wait-free*: **cada** hilo termina en un número acotado de pasos propios; ejemplo: `fetch_and_add`.
- Por qué importa: la gestión de memoria está en todas partes; si una lectura puede atascarse, todo puede.

Diap. 11: Lo que había

- Recorrer la tabla por columnas: nadie cumplía las cuatro cosas (*wait-free*, sin memoria extra, hardware actual, rápido).
- Sundell es *wait-free* pero cada CAS cuesta $\Theta(P)$; Detlefs necesita DCAS; Anderson et al. usan reclamación diferida (memoria extra); *split reference count* es *lock-free*.

Diap. 12: Contribuciones

- Cinco puntos: *wait-free* `load`/`store`/`swap` sin memoria extra; `store` O(1) y `load` casi siempre una instrucción (peor caso O(P)); CAS y *aliasing* solo *lock-free*; reexplica el *split refcount* como conteo ponderado; experimentos.

Diap. 13: Demo 1 — la carrera existe (AddressSanitizer)

- Arriba (captura grande): la versión ingenua con 8 hilos. Señalar la línea roja `ERROR: AddressSanitizer: heap-use-after-free`.
- Señalar las dos trazas: `WRITE ... Ingenua::load()` (el `fetch_add` sobre el libro) y `freed by thread T2 ... Ingenua::store()`: justo lo que dibujamos en la diapositiva 8.
- Abajo (más pequeña): el código original de los autores, mismas condiciones: 1 600 000 operaciones, 8 hilos, 56 ms, «sin errores de memoria».
- Si hay terminal abierta: lanzar la ingenua; puede tardar varias ejecuciones en fallar (ventana de nanosegundos).

Diap. 14: La idea, antes de pasar a Hugo

- Dibujo: el cartel guarda (libro A, talonario 104); la hoja de A vale 106 = 104 + 1 + 1 (dos pases).
- Los tickets ya están pagados en la hoja. Leer = arrancar un ticket con **un** `fetch_and_add`, que a la vez dice qué libro hay.
- **Transición:** «Leer y apuntarse son ahora un solo paso. Cómo se mantiene esa cuenta correcta lo explica Hugo.»

Parte 2 — Hugo: la solución

Diap. 15 (portadilla) — «Gracias, David. Voy a enseñar el conteo ponderado y los dos algoritmos.»

Diap. 16: Conteo de referencias ponderado

- Cada referencia tiene peso; el contador es la suma de los pesos. Clonar = repartir peso **sin tocar el contador**.
- Seguir la tabla: X = 10 ⟶ clonar X en Y (X = 6, Y = 4, hoja no se toca) ⟶ tirar Y (hoja 6) ⟶ clonar ⟶ tirar Z y X ⟶ hoja 0, se libera.
- Idea: en memoria compartida, no tocar la hoja (una posición lejana) es lo que elimina la carrera.

Diap. 17: El peso en el atomic shared pointer

- Fig. 2: (a) cartel con 104, dos pases, hoja 106. (b) tras un `load`: cartel 103, un pase nuevo, hoja sigue en 106.
- Al crear el cartel se le da un peso $C \gg P$ y se suma C a la hoja; cada pase pesa 1.

- Invariante: hoja ≥ peso del cartel + pases. Por arriba es seguro; por abajo se liberaría con lectores dentro.

Diap. 18: Algoritmo 1: `store` y `load`

- `store`: (1) sumar C al objeto nuevo *antes* de publicarlo; (2) `swap`: pone (nuevo, C) y se lleva (viejo, w) de golpe; (3) restar al viejo **solo el sobrante w**.
- `load`: un `fetch_and_add` de −1 ticket que devuelve el valor anterior; si quedan menos de P, recarga (algoritmo 2); devuelve un pase de peso 1, ya pagado.
- Insistir: el viejo **se libera cuando le toca**, no se retrasa.

Diap. 19: Por qué desaparece la carrera

- Solo hay dos órdenes posibles porque F (el `fetch_and_add` de Ana) y S2 (el `swap` de Beto) tocan la misma palabra.
- Caso 1: F antes que S2 ⟶ Ana tiene un pase de A y la hoja deja 1 para ella. Caso 2: F después ⟶ Ana se lleva un pase de B (ya respaldado) y nunca toca A.
- «No hay tercer caso»: el estado «he leído pero no me he apuntado» ya no existe.

Diap. 20: Empaquetar puntero y peso en 64 bits

- x86-64 usa 48 bits de dirección: 16 bits altos libres para el peso ($C \le 2^{16}-1$); con bits de alineación, 19.
- Restar un ticket es restar 2^{48} a la palabra entera: la dirección no cambia. Mostrar la resta hexadecimal.
- Con `null`, el peso da la vuelta sin estropear nada.

Diap. 21: Recargar el talonario: el punto delicado

- Cuando el peso baja de P, *ese hilo* recarga: reserva C − w en la hoja y sube el peso.
- Dos trampas: el cartel pudo cambiar de libro; otro hilo pudo recargar ya. Por eso hace falta un **CAS**, no un `fetch_and_add`.
- R-1: $C \ge 2P$. R-2: el peso solo baja de 1 en 1 o se fija por encima de P. Consecuencia: si el CAS falla y el peso subió, alguien ya recargó ⟶ **salida temprana**.

Diap. 22: La idea informal (Figura 4)

- Seguir los números 1–4 sobre el gráfico: uno empieza a recargar; otro recarga antes; el CAS del primero falla pero el peso vuelve a ser bajo y *mayor que el que vio* ⟶ puede irse.
- Frase: «quien mete el talonario en la zona roja, lo saca; si otro se adelanta, se va.»

Diap. 23: Algoritmo 2: `increase_weight`

- Cinco pasos: reservar C − w en la hoja *antes* del CAS; CAS; éxito ⟶ terminar; otro puntero o peso mayor ⟶ devolver la reserva y salir; peso menor ⟶ reintentar.
- Los pesos esperados forman una cadena estrictamente decreciente entre P−1 y 0: como mucho P intentos.

Diap. 24: Ejemplo completo, P = 4, C = 8

- Ir fila a fila: cuatro lecturas normales (8 ⟶ 4); Ana y Beto «ofenden» (bajan de P) y recargan; Ana gana el CAS (2 ⟶ 7); Beto falla, ve 7 > 2 y **sale**, devolviendo su reserva.
- Cierre: 7 tickets + 6 pases = 13 = hoja. La hoja nunca va por debajo.

Diap. 25: Demo 2 (1/2) — el simulador

- Qué es: simulador de intercalados, paso atómico a paso atómico, que comprueba los invariantes tras cada paso (20 000 ejecuciones por variante).
- Señalar la tabla: **Paper 0 fallos**; Ingenua 4 796 (uso tras liberar, 6 pasos); Umbral 0: 5 089 (peso negativo, 2 pasos); Sin reserva: 1 171 (liberación prematura, 6 pasos); Sin salida temprana: 5 467 (desborde, 7 pasos).
- Abajo: validación con 4 hilos, 0 fallos; intentos de CAS 1: 80,1 %, 2: 18,2 %, 3: 1,7 %, 4: 0,06 % (la cota es P = 4).

Diap. 26: Demo 2 (2/2) — traza de un contraejemplo

- Leer la traza: Beto crea L2 con reserva, Carla hace `load` (ticket 3 ⟶ 2, es ofensora y le toca recargar), Beto hace `swap` y resta la reserva a L1: L1 queda a 0 con Carla todavía dentro. Línea roja: «LIBERACIÓN PREMATURA».
- Moraleja: cada pieza del algoritmo evita un fallo concreto; con quitar una, el simulador lo encuentra en pocos pasos.
- **Transición:** «Esto no es una demostración, es evidencia. La demostración y la evaluación las explica Diego.»

Parte 3 — Diego: corrección, evaluación y conclusiones

Diap. 27 (portadilla) — «Gracias, Hugo. Veré por qué es correcto, cuánto cuesta y qué medimos.»

Diap. 28: La demostración: el peso nunca es negativo

- Por qué importa: el peso es de 16 bits sin signo; −1 sería 65 535 *tickets fantasma* sin respaldo ⟶ *use-after-free*.
- Ofensa: decrementar el peso cuando vale ≤ P. O: multiconjunto de ofensores.
- Invariante 1: cada hilo está en O como mucho una vez. Invariante 2: peso ≥ P − |O|. Luego peso ≥ P − |O| ≥ P − P = 0.
- Por eso el umbral es P y no 0: es el colchón para que todos ofendan a la vez.

Diap. 29: Coste

- El bucle CAS repite solo si otro hilo bajó el peso: cadena $w_0 > w_1 > \ldots \geq 0$ con $w_0 \leq P-1$ ⟶ como mucho P intentos.
- `load`: una instrucción casi siempre, **O(P) en el peor caso**; `store`/`swap` O(1). CAS y *aliasing* sin cota (*lock-free*).
- Amortizado: con $C = 2^{19}$ y $P = 2^{16}$ hay ≈ 458 752 lecturas entre recargas.

Diap. 30: Extensiones

- `null`, punteros débiles (misma técnica), compare-and-set (bucle CAS; las lecturas hacen fallar al escritor ⟶ *lock-free*) y *aliasing* (dos punteros en 128 bits).

- Señalar el Algoritmo 3 solo para mostrar que es un bucle CAS.

Diap. 31: Experimentos del paper: montaje

- C = 2^{19}, P = 2^{16}. Rendimiento: 4 × Xeon E7-8867, 144 hilos. Latencia: Xeon w7-3465X, 56 hilos.
- Banco: pila concurrente con n pilas y u % de actualizaciones. Rivales: Libstdc++ (candados), Williams y Folly (*split refcount*), Anderson (*hazard pointers*).

Diap. 32: Figura 5: rendimiento

- Con 1 hilo es la más rápida en todos los paneles; gana a los que no usan memoria extra (≈ 7× Williams y ≈ 20× Folly con 1 pila y 144 hilos).
- Frente a Anderson depende de la contención: pierde con una sola pila porque las actualizaciones usan el CAS *lock-free*.

Diap. 33: Figura 6: latencia

- Con p = 56, mediana de `load`: Ours 1,14 µs, Anderson 2,58, Folly 11,6, Libstdc++ 99,7 µs. Además tiene la cola más corta.
- `store`: 2.º con p = 1 y 1.º con p = 56.

Diap. 34: Pruebas en local (1): el boceto

- Captura de arriba: el boceto con P = 8, C = 16; versión del paper **0 errores**, máx. 7 intentos de CAS (≤ P). Ingenua: 601 681 errores y −102 892 «sin liberar» = **dobles liberaciones**.
- Captura de abajo: sin salida temprana, 0 errores pero **38 intentos > P** (deja de ser *wait-free*); con C = P (rompe R-1): 50 060 errores y 3 055 sin liberar.

Diap. 35: Pruebas en local (2): rendimiento

- Máquina: i5-10400, 12 hilos, WSL, g++ 13.3; 1 s por punto. Autores = código original; Libstdc++ = `atomic<shared_ptr>` de GCC; Mutex = `shared_ptr` + `std::mutex`.
- n = 1, u = 1: 34,3 / 2,2 / 6,4 Mop/s. n = 10, u = 1: 45,8 / 7,0 / 19,1. n = 10^5, u = 50: 29,8 / 19,6 / 17,4.
- **Matiz honesto:** con n = 1, u = 50 el mutex gana (2,9 frente a 2,2): las actualizaciones usan el CAS *lock-free* que reintenta, como reconoce el paper. Con 1 hilo, Autores es la más rápida en los seis paneles.

Diap. 36: Pruebas en local (3): latencia con 12 hilos

- `load` (mediana / p99): Autores 269 ns / 0,83 µs; Libstdc++ 3,06 µs / 16,0 µs; Mutex 61 ns / 27,6 µs. `store`: 209 ns / 0,69 µs; 3,05 / 15,8 µs; 152 ns / 30,7 µs.
- Mensaje: **la mediana engaña, la cola delata**: el mutex gana en mediana y pierde en p99 (27,6 µs).
- Nota de método: con 1 hilo cada muestra es la media de 16 operaciones (el reloj cuesta ~46 ns, más que la operación).

Diap. 37: ¿Coincide con el paper?

- Se reproduce: 0 errores con ASan, con 1 hilo la más rápida, gana a GCC, mejor cola que el mutex.
- Matices: el mutex la supera con 1 pila y 50 % de actualizaciones; 12 hilos y no 144; sin Folly/Williams/Anderson; un solo equipo y una ejecución ⟶ tendencias, no cifras absolutas.

Diap. 38: Relación con el split reference count

- *Split refcount*: contador local que sube desde 0; el paper cuenta lo contrario: peso = C − contador local.
- Lo nuevo no es la técnica sino la recarga que **sale si otro ya recargó**: eso la hace *wait-free*. Nota: Folly pasa a bloqueante con más de C hilos.

Diap. 39: Valoración crítica

- Fortalezas: simple (≈ 30 líneas de núcleo), *wait-free*, sin memoria extra, liberación inmediata, lecturas de una instrucción.
- Límites: CAS y *aliasing* solo *lock-free*; cada lectura **escribe** en la variable compartida; límite de P hilos (de corrección); depende de bits libres del puntero.

Diap. 40: Conclusiones

- (1) Conteo de referencias *wait-free* sin retrasar la liberación. (2) Clave: peso prepagado en el puntero. (3) Recarga ≤ P intentos por R-1 y R-2. (4) Lo comprobamos en local.
- Para discutir: ¿CAS *wait-free* con ayuda entre hilos?, ¿modo de lectura sin escritura?

Diap. 41: ¿Preguntas? — Pasar el turno de palabra a quien corresponda según el tema (ver preguntas probables).

Preguntas probables y respuestas cortas

Parte 1 (David)

1. **¿Por qué no basta con un candado?** Funciona, pero es bloqueante: un hilo parado con el candado frena a todos. En nuestras pruebas la versión con mutex tiene mediana baja pero p99 de 27,6 µs.
2. **¿Por qué no un DCAS?** Necesitaríamos modificar dos posiciones separadas (cartel y hoja) a la vez; el hardware actual no lo tiene.
3. **¿Con qué frecuencia ocurre la carrera?** Rarísimo, ventana de nanosegundos; por eso se cuela en producción. Con AddressSanitizer y 8 hilos la ingenua falla enseguida.
4. **¿Qué diferencia hay entre lock-free y wait-free?** Lock-free: siempre *algún* hilo avanza. Wait-free: *cada* hilo termina en un número acotado de pasos. Ejemplo: bucle CAS frente a `fetch_and_add`.
5. **¿Por qué no usar punteros de riesgo o reclamación diferida?** Funcionan, pero gastan memoria extra y retrasan la liberación (el rc = 1 deja de ser fiable).
6. **¿Qué es el shared_ptr atómico de GCC?** `std::atomic<std::shared_ptr>` en libstdc++ usa candados internos: es correcto pero bloqueante (en la tabla, 3,06 µs de mediana de `load` con 12 hilos).
7. **¿Esto afecta a Rust o Swift?** El problema es el mismo en cualquier lenguaje con conteo de referencias y variables compartidas mutables.
8. **¿Por qué importa liberar de inmediato?** Permite modificar en el sitio cuando rc = 1 y evita picos de memoria.

Parte 2 (Hugo)

1. **¿Qué es el «peso» con la analogía?** El talonario de tickets del cartel: tickets ya apuntados en la hoja del libro.
2. **¿Por qué desaparece la carrera?** Porque leer y apuntarse son el mismo `fetch_and_add`: no existe el estado «he leído pero no me he apuntado».
3. **¿Por qué hace falta C ≥ 2P (R-1)?** Para que tras recargar el peso quede por encima de P aun con P hilos ofendiendo: 0 + 2P − (P−1) = P+1. Con C = P, el boceto dio 50 060 errores.
4. **¿Para qué sirve la salida temprana (R-2)?** Si el CAS falla y el peso subió, otro ya recargó: salimos y devolvemos la reserva. Sin ella, 38 intentos con P = 8 (no es *wait-free*).
5. **¿Y si el cartel cambia de libro durante la recarga?** El CAS compara también el puntero: falla, se devuelve la reserva y se sale.
6. **¿Por qué se reserva antes del CAS?** Para que la hoja ya cubra los tickets cuando se publiquen; sin reserva el simulador halla liberación prematura (1 171 fallos).
7. **¿Caben puntero y peso en 64 bits?** Sí en x86-64: 48 bits de dirección y 16 libres (19 con alineación).
8. **¿Qué mostró el simulador?** Paper 0 fallos en 20 000 ejecuciones; cada variante rota falla en 2 a 7 pasos.

Parte 3 (Diego)

1. **¿Cuánto cuesta un load?** Normalmente una instrucción atómica; peor caso O(P) por la recarga, lo que no ocurre casi nunca (≈ 2 por millón con $C = 2^{19}$, $P = 2^{16}$).
2. **¿Por qué el umbral es P y no 0?** Es el colchón: aunque los P hilos ofendan a la vez, peso ≥ P − |O| ≥ 0.
3. **¿Qué pasa si hay más de P hilos?** Se pierde la garantía de corrección (no de rendimiento); P es el máximo de hilos que usan esa variable.
4. **¿Por qué en local el mutex gana a veces?** Con 1 pila y 50 % de actualizaciones, el CAS *lock-free* reintenta (2,9 frente a 2,2 Mop/s); el paper también lo reconoce.
5. **¿Por qué no repetisteis con 144 hilos?** Nuestro equipo tiene 12; las tendencias coinciden, no las cifras absolutas.
6. **¿Por qué mediana y p99?** La mediana engaña (Mutex 61 ns), la cola delata a quien espera (27,6 µs).
7. **¿Cuál es la diferencia con el split reference count?** Contador local frente a peso: peso = C − contador local; la novedad es la recarga con salida temprana.
8. **¿Limitaciones?** CAS y *aliasing* solo *lock-free*; las lecturas escriben; dependen de bits libres del puntero (5 niveles de paginación, ARM con PAC/MTE).

Errores típicos que evitar

- «**El algoritmo retrasa la liberación**»: falso. Libera inmediatamente cuando el contador llega a 0; ese es su punto frente a la reclamación diferida.
- «**Es O(P) siempre**»: O(P) es el **peor caso**; lo habitual es una instrucción atómica.
- «**Todo es wait-free**»: solo `load`, `store` y `swap`; CAS y *aliasing* son *lock-free*.

- **«Lock-free es lo mismo que wait-free»**: no; en lock-free un hilo concreto puede fallar siempre.
- **«Hemos medido en 144 hilos»**: no; nosotros 12 hilos en un i5-10400 con WSL. Los 144 hilos son del paper.
- **«El mutex es peor en todo»**: con n = 1, u = 50 gana; y en mediana de `load` (61 ns). Perder está en la cola.
- **«Reproducimos Folly/Williams/Anderson»**: no; solo Autores, Libstdc++ y Mutex.
- **Confundir ofensa con error**: ofender es bajar el peso estando en la zona roja; no es un fallo, solo obliga a recargar.
- **Decir que la ventana de la carrera es larga**: dura nanosegundos; por eso es difícil de ver.
- **Cifras sueltas**: citar siempre con su contexto (hilos, n, u), sobre todo los 34,3 Mop/s.